Working on Snowflake with Custom Dataset

1. Creating Database:

```
CREATE DATABASE IF NOT EXISTS SALES_DB;
```

2. Creating a Warehouse:

```
CREATE WAREHOUSE IF NOT EXISTS SALES_WH
WAREHOUSE_SIZE = XSMALL
AUTO_SUSPEND = 60
AUTO_RESUME = TRUE
INITIALLY_SUSPENDED = TRUE;
```

3. Creating a Schema:

```
CREATE SCHEMA IF NOT EXISTS SALES_DB.RAW;
USE SCHEMA SALES_DB.RAW;
```

4. Creating the necessary Tables:

```
CREATE TABLE IF NOT EXISTS SALES_DB.RAW.CUSTOMERS (
    CUSTOMER_ID VARCHAR(20),
    CUSTOMER_NAME VARCHAR(100),
    CITY VARCHAR(50),
    CUSTOMER_TYPE VARCHAR(30)
);

CREATE TABLE IF NOT EXISTS SALES_DB.RAW.PRODUCTS (
    PRODUCT_ID VARCHAR(20),
    PRODUCT_NAME VARCHAR(100),
    UNIT_PRICE VARCHAR(20),
    CATEGORY VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS SALES_DB.RAW.SALES (
    SALES_ID VARCHAR(20),
    CUSTOMER_ID VARCHAR(20),
    PRODUCT_ID VARCHAR(20),
    SALES_DATE VARCHAR(30),
    QUANTITY VARCHAR(20),
    CITY VARCHAR(50),
    PRODUCT VARCHAR(100),
    UNIT_PRICE VARCHAR(20)
);
```

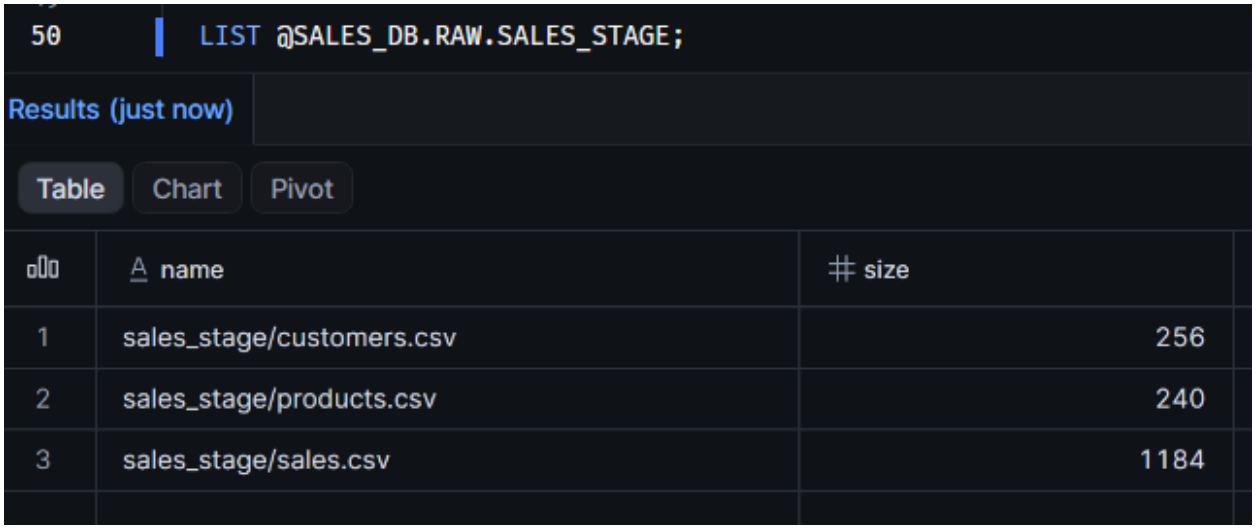
5. Creating a File Format:

```
CREATE OR REPLACE FILE FORMAT SALES_DB.RAW.CSV_FORMAT
    TYPE = CSV
    FIELD_DELIMITER = ','
    SKIP_HEADER = 1
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    EMPTY_FIELD_AS_NULL = TRUE
    NULL_IF = ('', 'NULL', 'null');
```

6. Creating a Stage to load data:

```
CREATE OR REPLACE STAGE SALES_DB.RAW.SALES_STAGE
    FILE_FORMAT = SALES_DB.RAW.CSV_FORMAT;
```

7. After loading Data into the Stage:



8. Loading Data into Tables:

We use the COPY INTO command to load individual files from the stage to each table.

```
COPY INTO SALES_DB.RAW.CUSTOMERS
FROM @SALES_DB.RAW.SALES_STAGE/customers.csv
FILE_FORMAT = (
    FORMAT_NAME = SALES_DB.RAW.CSV_FORMAT
)
ON_ERROR = 'ABORT_STATEMENT';

COPY INTO SALES_DB.RAW.PRODUCTS
FROM @SALES_DB.RAW.SALES_STAGE/products.csv
```

```
FILE_FORMAT = (
    FORMAT_NAME = SALES_DB.RAW.CSV_FORMAT
)
ON_ERROR = 'ABORT_STATEMENT';

COPY INTO SALES_DB.RAW.SALES
FROM @SALES_DB.RAW.SALES_STAGE/sales.csv
FILE_FORMAT = (
    FORMAT_NAME = SALES_DB.RAW.CSV_FORMAT
)
ON_ERROR = 'ABORT_STATEMENT';
```

9. Creating Cleaned Table:

```
CREATE OR REPLACE TABLE SALES_DB.RAW.CLEAN_CUSTOMERS AS
SELECT
    CUSTOMER_ID,
    COALESCE(NULLIF(TRIM(CUSTOMER_NAME), ''),'UNKNOWN') AS CUSTOMER_NAME,
    UPPER(NULLIF(TRIM(CITY), '')) AS CITY,
    UPPER(NULLIF(TRIM(CUSTOMER_TYPE), '')) AS CUSTOMER_TYPE
FROM SALES_DB.RAW.CUSTOMERS;
```



	CUSTOMER_ID	CUSTOMER_NAME	CITY	CUSTOMER_TYPE
1	C001	Arun Kumar	CHENNAI	RETAIL
2	C002	Priya Sharma	CHENNAI	WHOLESALE
3	C003	Rahul Nair	BENGALURU	RETAIL
4	C004	Meena Iyer	COIMBATORE	CORPORATE
5	C005	Karthik Raj	null	RETAIL
6	C006	Divya Menon	HYDERABAD	WHOLESALE

Analytical Queries:

1. Displaying all Sales:

```
83  SELECT * FROM SALES_DB.RAW.SALES;
```

Results (2 minutes ago) — Table | Chart | Pivot — 20 rows — 90ms

	SALES_ID	CUSTOMER_ID	PRODUCT_ID	SALES_DATE	QUANTITY	CITY	PRODUCT	UNIT_PRICE
	S001 5.0% S002 5.0% +18 more	C001 20.0% C002 20.0% +4 more	P001 20.0% P004 20.0% +4 more	2026-10-01 5.0% 2026-10-02 5.0% +18 more	1 40.0% 2 35.0% +3 more	Chennai 45.0% Bengaluru 15.0% +4 more	Laptop 20.0% Monitor 20.0% +5 more	12000.00 20.0% 55000.00 20.0% +4 more
1	S001	C001	P002	2026-10-01	2	Chennai	Wireless Mouse	850.00
2	S002	C002	P001	2026-10-02	1	Chennai	Laptop	55000.00
3	S003	C003	P003	2026-10-03	3	Bengaluru	Keyboard	1500.00
4	S004	C004	P004	2026-10-04	2	Coimbatore	Monitor	12000.00
5	S005	C005	P005	2026-10-05	1	null	USB-C Hub	2500.00
6	S006	C006	P006	2026-10-06	2	Hyderabad	Headphones	3200.00
7	S007	C001	P001	2026-10-07	1	Chennai	Laptop	55000.00
8	S008	C002	P002	2026-10-08	4	Chennai	Wireless Mouse	850.00
9	S009	C003	P004	2026-10-09	1	Bengaluru	Monitor	12000.00
10	S010	C004	P003	2026-10-10	2	Coimbatore	Keyboard	1500.00

2. Find Sales based on City:

```
85  SELECT * FROM SALES_DB.RAW.SALES
86  WHERE UPPER(TRIM(CITY)) = 'CHENNAI';
```

Results (just now) — Table | Chart | Pivot — 10 rows — 592ms

	SALES_ID	CUSTOMER_ID	PRODUCT_ID	SALES_DATE	QUANTITY	CITY	PRODUCT	UNIT_PRICE
	S001 10.0% S002 10.0% +8 more	C001 40.0% C002 40.0% +1 more	P001 30.0% P002 20.0% +4 more	2026-10-01 10.0% 2026-10-02 10.0% +8 more	1 40.0% 2 40.0% +1 more	Chennai 90.0% Chennai 10.0%	Laptop 30.0% Headphones 20.0% +4 more	55000.00 30.0% 3200.00 20.0% +4 more
1	S001	C001	P002	2026-10-01	2	Chennai	Wireless Mouse	850.00
2	S002	C002	P001	2026-10-02	1	Chennai	Laptop	55000.00
3	S007	C001	P001	2026-10-07	1	Chennai	Laptop	55000.00
4	S008	C002	P002	2026-10-08	4	Chennai	Wireless Mouse	850.00
5	S011	C005	P001	2026-10-11	1	Chennai	Laptop	55000.00
6	S013	C001	P006	2026-10-13	1	Chennai	Headphones	3200.00
7	S014	C002	P004	2026-10-14	2	Chennai	Monitor	12000.00
8	S017	C005	P003	2026-10-17	2	Chennai	Keyboard	1500.00
9	S019	C001	P005	2026-10-19	2	Chennai	USB-C Hub	2500.00
10	S020	C002	P006	2026-10-20	null	Chennai	Headphones	3200.00

3. Calculate Revenue per Sale:

```sql
SELECT
    SALES_ID,
    PRODUCT,
    QUANTITY,
    UNIT_PRICE,
    QUANTITY * UNIT_PRICE AS REVENUE
FROM SALES_DB.RAW.SALES;
```

Results (just now)

Table | Chart | Pivot | Next steps | 20 rows | 491ms

	SALES_ID	PRODUCT	QUANTITY	UNIT_PRICE	REVENUE
	S001 5.0% S002 5.0% +18 more	Laptop 20.0% Monitor 20.0% +5 more	1 40.0% 2 35.0% +3 more	12000.00 20.0% 55000.00 20.0% +4 more	1700 55000
1	S001	Wireless Mouse	2	850.00	1700
2	S002	Laptop	1	55000.00	55000
3	S003	Keyboard	3	1500.00	4500
4	S004	Monitor	2	12000.00	24000
5	S005	USB-C Hub	1	2500.00	2500
6	S006	Headphones	2	3200.00	6400
7	S007	Laptop	1	55000.00	55000
8	S008	Wireless Mouse	4	850.00	3400
9	S009	Monitor	1	12000.00	12000
10	S010	Keyboard	2	1500.00	3000

4. Calculate Product Wise Sales:

```sql
SELECT
    TRIM(PRODUCT) AS PRODUCT,
    SUM(QUANTITY * UNIT_PRICE) AS TOTAL_SALES
FROM SALES_DB.RAW.SALES
GROUP BY TRIM(PRODUCT)
ORDER BY TOTAL_SALES DESC;
```

Results (just now)

Table | Chart | Pivot | 6 rows | 312ms

	PRODUCT	TOTAL_SALES
1	Laptop	220000
2	Monitor	72000
3	USB-C Hub	15000
4	Keyboard	10500
5	Headphones	9600
6	Wireless Mouse	9350

5. Joining Sales and Customer Table:

```
SELECT
    S.SALES_ID,
    C.CUSTOMER_NAME,
    C.CUSTOMER_TYPE,
    S.PRODUCT_ID,
    S.PRODUCT,
    S.QUANTITY,
    S.UNIT_PRICE,
    S.QUANTITY * S.UNIT_PRICE AS TOTAL_AMOUNT
FROM SALES_DB.RAW.SALES S
JOIN SALES_DB.RAW.CLEAN_CUSTOMERS C
    ON S.CUSTOMER_ID = C.CUSTOMER_ID;
```

Open or move CoCo

Results (just now)

Table | Chart | Pivot | Next steps | 20 rows | 600ms

	SALES_ID	CUSTOMER_NAME	CUSTOMER_TYPE	PRODUCT_ID	PRODUCT	QUANTITY	UNIT_PRICE	TOTAL_AMOUNT
	S001 5.0% S002 5.0% +18 more	Arun Kumar 20.0% Priya Sharma 20.0% +4 more	RETAIL 50.0% WHOLESALE 35.0% +1 more	P001 20.0% P004 20.0% +4 more	Laptop 20.0% Monitor 20.0% +5 more	1 40.0% 2 35.0% +3 more	12000.00 20.0% 55000.00 20.0% +4 more	1700 55000
1	S001	Arun Kumar	RETAIL	P002	Wireless Mouse	2	850.00	1700
2	S002	Priya Sharma	WHOLESALE	P001	Laptop	1	55000.00	55000
3	S003	Rahul Nair	RETAIL	P003	Keyboard	3	1500.00	4500
4	S004	Meena Iyer	CORPORATE	P004	Monitor	2	12000.00	24000
5	S005	Karthik Raj	RETAIL	P005	USB-C Hub	1	2500.00	2500
6	S006	Divya Menon	WHOLESALE	P006	Headphones	2	3200.00	6400
7	S007	Arun Kumar	RETAIL	P001	Laptop	1	55000.00	55000
8	S008	Priya Sharma	WHOLESALE	P002	Wireless Mouse	4	850.00	3400
9	S009	Rahul Nair	RETAIL	P004	Monitor	1	12000.00	12000

Feedback

6. Total Sales per Customer Type:

```
SELECT
    C.CUSTOMER_TYPE,
    SUM(S.QUANTITY * S.UNIT_PRICE) AS TOTAL_SALES
FROM SALES_DB.RAW.SALES S
JOIN SALES_DB.RAW.CLEAN_CUSTOMERS C
    ON S.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY C.CUSTOMER_TYPE
ORDER BY TOTAL_SALES DESC;
```

Results (just now)

Table | Chart | Pivot | 3 rows | 386ms

	CUSTOMER_TYPE	TOTAL_SALES
1	RETAIL	146150
2	WHOLESALE	108300
3	CORPORATE	82000

Query Optimization

For query optimization, we shall generate a dataset with a large amount of data entries.

1. Generating the Dataset:

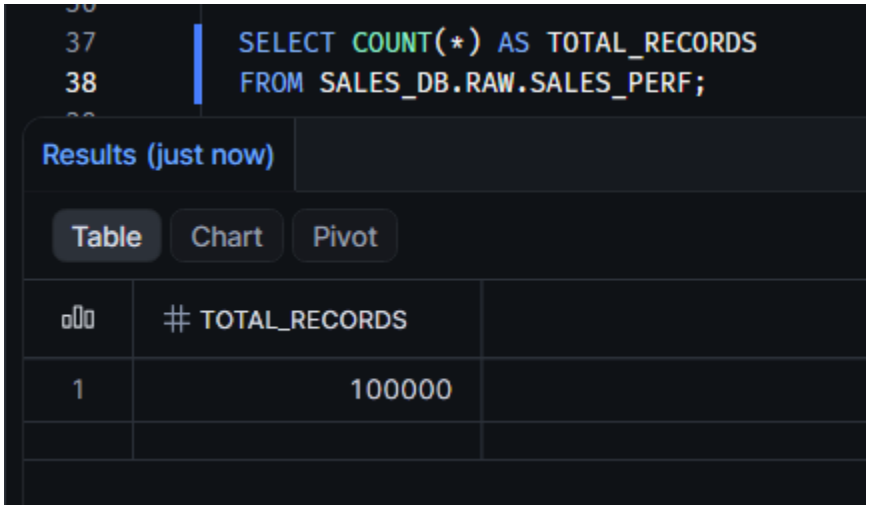
```
CREATE OR REPLACE TABLE SALES_DB.RAW.SALES_PERF AS
WITH GENERATED_DATA AS (
    SELECT
        SEQ4() + 1 AS ORDER_ID,
        UNIFORM(1, 10000, RANDOM()) AS CUSTOMER_ID,
        'Product-' || UNIFORM(1, 50, RANDOM()) AS PRODUCT,
        UNIFORM(1, 5, RANDOM()) AS QUANTITY,
        UNIFORM(100, 5000, RANDOM())::NUMBER(10,2) AS UNIT_PRICE,
        DATEADD(
            DAY,
            -UNIFORM(0, 730, RANDOM()),
            CURRENT_DATE()
        ) AS ORDER_DATE,
        UNIFORM(1, 5, RANDOM()) AS CITY_CODE
    FROM TABLE(GENERATOR(ROWCOUNT => 100000))
)

SELECT * FROM SALES_DB.RAW.SALES_PERF;
```

Results (just now)

Table | Chart | Pivot | Quick insights | Next steps | 100,000 rows | 799ms

	ORDER_ID	CUSTOMER_ID	PRODUCT	QUANTITY	UNIT_PRICE	ORDER_DATE	CITY
1	1	7540	Product-32	3	845.00	2025-12-17	Bangalore
2	2	5009	Product-32	2	1668.00	2024-11-26	Chennai
3	3	2480	Product-7	1	1594.00	2025-08-08	null
4	4	9690	Product-6	1	2560.00	2024-12-03	Mumbai
5	5	890	Product-12	5	3553.00	2026-04-03	Bangalore
6	6	4067	Product-22	3	2068.00	2025-10-01	Chennai
7	7	4041	Product-8	3	1311.00	2026-07-20	Mumbai



2. Executing Baseline Queries (without optimization):

We are executing 4 different queries:

1. Total revenue across all orders
2. Revenue by city
3. Product-wise revenue
4. Filter sales by city

The resultant performance metrics are as follows:

Query	Operation	Execution time	Bytes scanned	Partitions scanned
Q1	Total revenue	480 ms	0.30 MB	1 / 1
Q2	Revenue by city	226 ms	0.38 MB	1 / 1
Q3	Product-wise revenue	114 ms	0.41 MB	1 / 1
Q4	Chennai filter	769 ms	1.03 MB	1 / 1

3. Runtime after Query Optimization:

We perform optimization strategies to the queries and execute them again. By comparing the query profile, we will be able to see the differences in runtime.

Query	Operation	Before	After	Time saved
Q2	Revenue by city	226 ms	62 ms	164 ms
Q3	Product-wise revenue	114 ms	50 ms	64 ms
Q4	Chennai filter	769 ms	626 ms	143 ms

Final Cleanup

1. Drop all Tables:

```
122  DROP TABLE SALES_DB.RAW.SALES_PERF;
123  DROP TABLE SALES_DB.RAW.SALES;
124  DROP TABLE SALES_DB.RAW.PRODUCTS;
125  DROP TABLE SALES_DB.RAW.CUSTOMERS;
126  DROP TABLE SALES_DB.RAW.CLEAN_CUSTOMERS;
```

2. Drop Stages and File Format:

```
128  DROP STAGE SALES_DB.RAW.SALES_STAGE;
129  DROP FILE FORMAT SALES_DB.RAW.CSV_FORMAT;
```

3. Drop Warehouse and Database:

```
131  DROP SCHEMA SALES_DB.RAW;
132  DROP WAREHOUSE SALES_WH;
133  DROP DATABASE SALES_DB;
```